REPORT · 2025-07-07

MCS v2.0 — Upgradation Specification

Management Control System · Upgrade from Google Sheets + Apps Script to Full-Stack Python + Supabase

AUTHOR

MCS Development Team

DATE

2025-07-07

Project: Management Control System
Current Version: v1.x (Single-file React + Google Sheets)
Target Version: v2.0 (Full-stack + AI + Multi-channel)
Date: July 2025
Status: Draft

Table of Contents

---

1. Executive Summary

The current MCS operates as a client-heavy React SPA backed by Google Sheets (read via public CSV, write via Apps Script) with a thin Render.com API for escalation emails. This architecture served well for rapid prototyping but hits hard limits at scale:

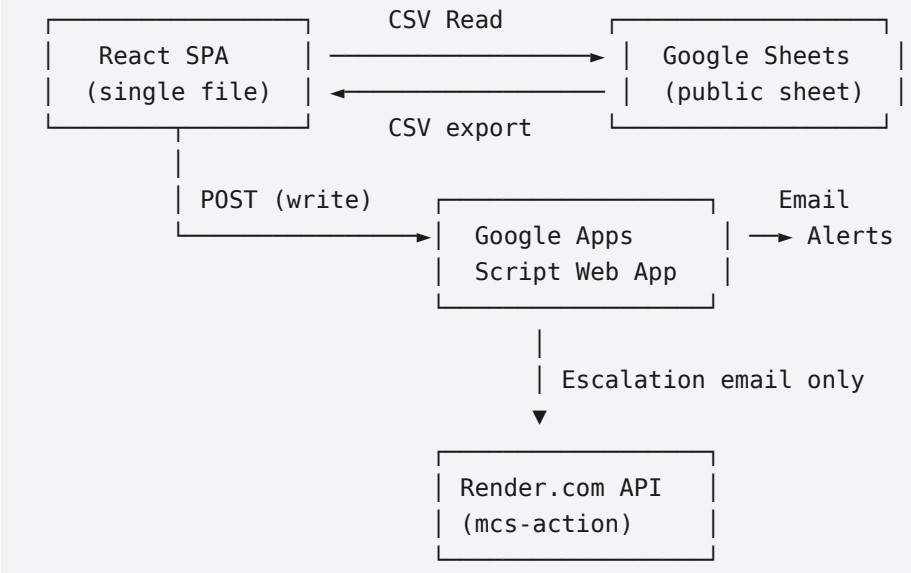
Pain Point	Current State	Target State
Speech-to-text	Chrome Web Speech API only (browser-dependent, no offline)	**Any STT engine** (Deepgram, Whisper, Azure, Google Cloud)
Notifications	In-app + email only (email has low open rates)	**WhatsApp Business API** as primary alert channel
Meeting Insights	Rule-based keyword matching	**AI-powered** summaries via Gemini/ChatGPT
Backend	Google Apps Script (write), Render.com (email only)	**Dedicated Python backend** (FastAPI)
Database	Google Sheets (concurrent write issues, no row-level locking)	**Supabase (PostgreSQL)** — real relational DB with RLS
Scalability	~50 concurrent users before Sheet throttling	**500+ concurrent users**
Offline support	None	Partial via service worker

MCS v2.0 retains the React frontend but replaces the entire backend/data layer and adds three new capability modules.

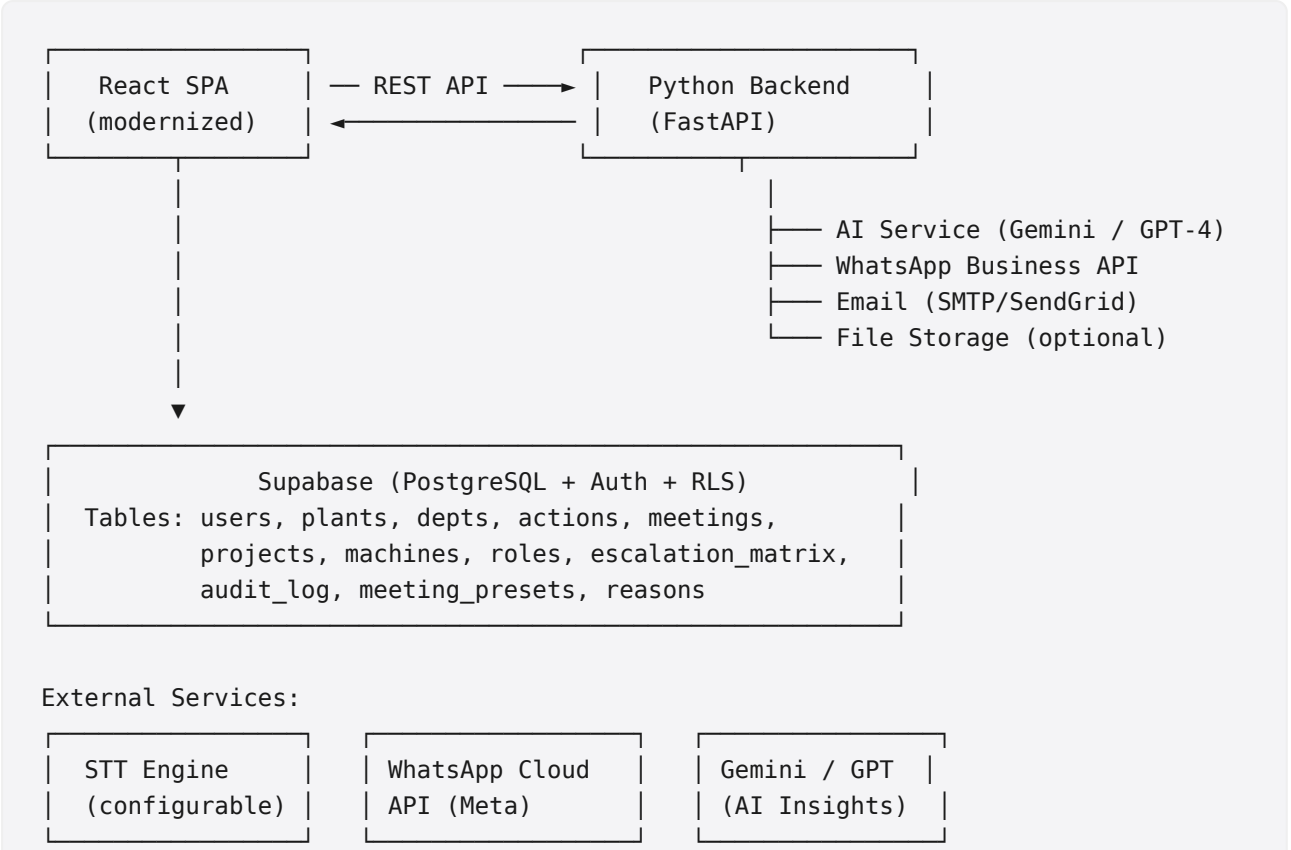
---

2. Architecture Overview

Current Architecture



Target Architecture (v2.0)



---

3. Upgrade Module 1 — Speech-to-Text Engine

3.1 Objective

Decouple STT from Chrome's built-in `SpeechRecognition` API and make it a pluggable service. Support Deepgram, OpenAI Whisper, Google Cloud Speech-to-Text, and Azure AI Speech — selected per-deployment via config.

3.2 Why

The current implementation uses:

```
// Current (MeetingRoom.jsx)
const recognition = new (window.SpeechRecognition || window.webkitSpeechRecognition)()
recognition.continuous = true;
recognition.interimResults = true;
recognition.lang = 'en-IN';
```

Problems:

- **Browser-only** — no mobile Safari, no Firefox support
- **No server-side fallback** — fails completely offline
- **No custom vocabulary** — can't be trained on domain terms (MPM, ferroalloy, EAF, etc.)
- **Quality variance** — depends on OS-level Chrome engine
- **No streaming to server** — all processing client-side

3.3 Proposed Design

Abstraction Layer — `STTService` interface:

```
# backend/stt/base.py
from abc import ABC, abstractmethod
from typing import AsyncIterator

class STTEngine(ABC):
    name: str  # "deepgram", "whisper", "google", "azure"

    @abstractmethod
    async def stream_transcribe(
        self, audio_chunk: bytes, language: str = "en-IN"
    ) -> AsyncIterator[str]:
        """Yield interim + final transcripts as audio chunks arrive."""
        pass

    @abstractmethod
    async def transcribe_file(self, audio_path: str, language: str = "en-IN") -> str:
        """Full file transcription (for post-meeting processing)."""
        pass
```

Concrete implementations:

Engine	Class	Notes
Deepgram	`DeepgramSTT`	Real-time streaming, excellent for Hindi-English mix, ~$0.004/min
Whisper (self-hosted or API)	`WhisperSTT`	Highest accuracy, supports 100+ languages, no streaming (file-based)
Google Cloud STT	`GoogleSTT`	Streaming, good accuracy, requires GCP setup
Azure AI Speech	`AzureSTT`	Streaming, good for enterprise, Azure account required

Frontend integration (MeetingRoom upgrade):

```
// Frontend streams audio to backend via WebSocket
// Backend proxies to selected STT engine

// Option A: Client-side WebSocket streaming
//   Microphone → MediaRecorder (blob chunks) → WebSocket → Backend → STT Engine
//   Backend streams transcripts back via same WebSocket

// Option B: Backend-side capture (no client code change)
//   Client keeps using Web Speech API → sends interim results to backend
//   Backend re-transcribes via premium engine for quality boost
//   Merges/replaces low-confidence segments
```

Recommended: Option A (WebSocket streaming) — decouples browser entirely.

3.4 Audio Flow (Option A)

```
Microphone (getUserMedia)
       │
       ▼
MediaRecorder (audio/webm, chunk every 500ms)
       │
       ▼
WebSocket client (frontend)
       │
       ▼ WebSocket (wss://api.mcs.com/stt/stream?meeting_id=xxx)
       │
       ▼
FastAPI WebSocket endpoint
       │
       ├──► STT Engine (Deepgram recommended — has native WebSocket support)
       │              │
       │              ▼
       │         Transcript chunks (interim → final)
       │              │
       │◄─────────────┘
       │
       ▼
Broadcast back to frontend WebSocket clients
       │
       ▼
MeetingRoom receives transcript lines (replaces current SpeechRecognition logic)
```

3.5 Configuration

```
# backend/config.py
STT_ENGINE = "deepgram"  # or "whisper", "google", "azure"
DEEPGRAM_API_KEY = os.getenv("DEEPGRAM_API_KEY")
WHISPER_API_URL = os.getenv("WHISPER_API_URL")
GOOGLE_STT_CREDENTIALS = os.getenv("GOOGLE_APPLICATION_CREDENTIALS")
AZURE_SPEECH_KEY = os.getenv("AZURE_SPEECH_KEY")
AZURE_SPEECH_REGION = "centralindia"
```

3.6 Acceptance Criteria

- [ ] Frontend can toggle STT engine via config without code changes
- [ ] Real-time transcripts appear in MeetingRoom with < 1s latency
- [ ] Works on Chrome, Firefox, Safari, mobile browsers
- [ ] Domain vocabulary (plant-specific terms) is handled correctly
- [ ] Graceful degradation if STT service is down (fallback message)
- [ ] STT usage is logged per-meeting for billing/cost tracking

---

4. Upgrade Module 2 — WhatsApp Alerting

4.1 Objective

Replace email-only escalation alerts with **WhatsApp Business API** as the primary notification channel. WhatsApp has 98% open rates vs ~20% for email in India.

4.2 Why

Current escalation alerts go via:

```
# Current: via Render.com API
POST https://mcs-action.onrender.com/api/email/escalate
```

Issues:

- Email is checked infrequently by plant floor staff
- Many workers don't have corporate email on their phones
- WhatsApp is the default communication channel in Indian manufacturing

4.3 WhatsApp Business API Options

Option	Provider	Cost	Setup Complexity	Recommended
WhatsApp Cloud API (Meta)	Meta	Per-message (₹0.30–₹0.70/msg)	Medium	✅ **Yes**
WhatsApp Business API (BSP)	Twilio, Gupshup, Kaleyra	₹0.20–₹1.00/msg + platform fee	High	For scale
Gupshup Enterprise	Gupshup	Per-message + platform fee	Low	Quick setup

Recommendation: Start with **WhatsApp Cloud API** (Meta's official API). Free tier: 1,000 notifications/month. Pay-as-you-go beyond that.

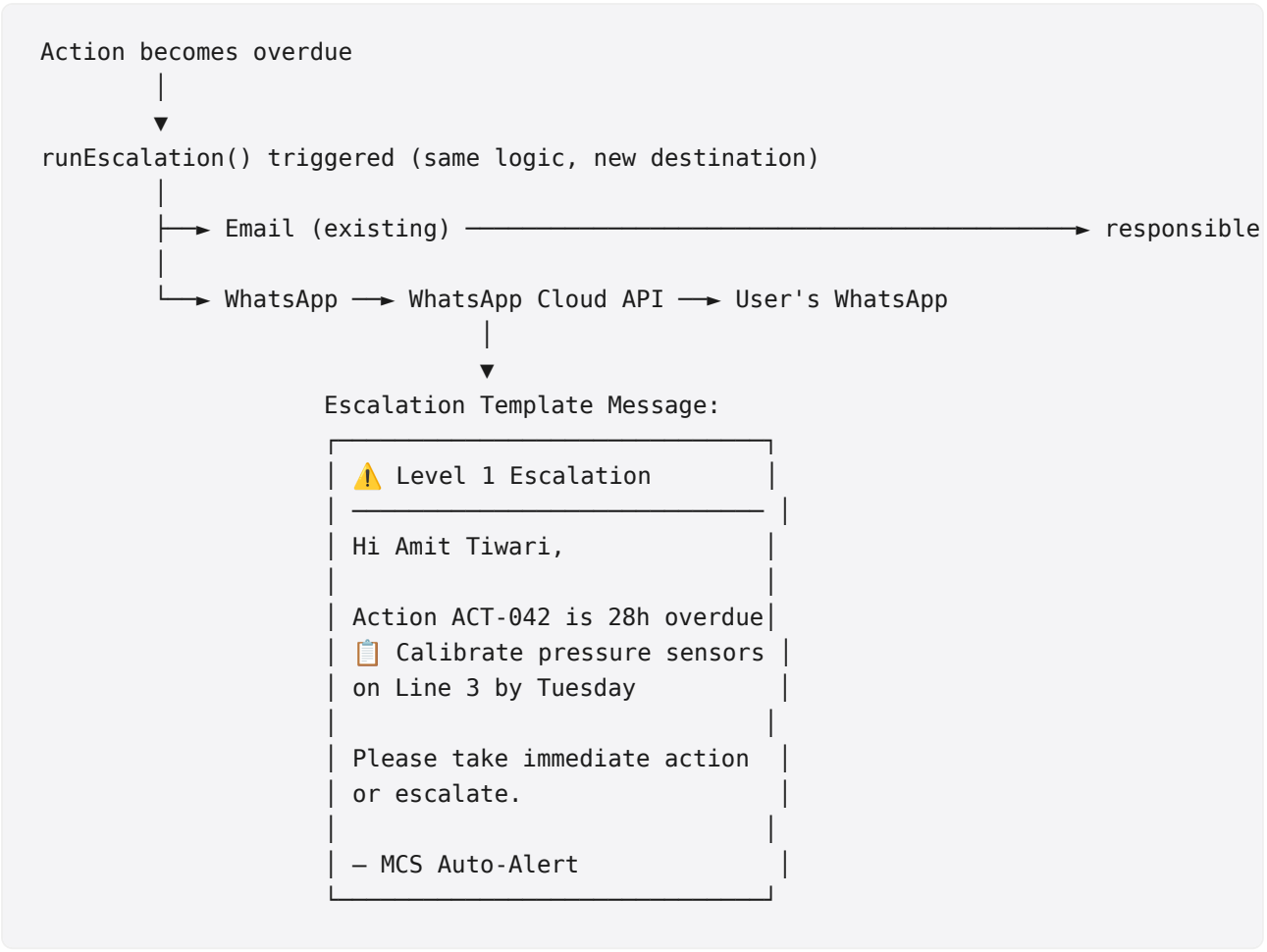
4.4 Proposed Design

Backend module: `backend/services/whatsapp.py`

```
from typing import Optional
import httpx

class WhatsAppService:
    """
    Sends WhatsApp alerts via Meta WhatsApp Cloud API.
    Supports: escalation alerts, action reminders, meeting reminders.
    """

    def __init__(self, phone_id: str, access_token: str, verified_number: str):
        self.phone_id = phone_id
        self.access_token = access_token
        self.verified_number = verified_number
        self.api_url = "https://graph.facebook.com/v18.0"

    async def send_template_message(
        self, to_number: str, template_name: str, header_vars: list, body_vars: list
    ) -> dict:
        """Send a pre-approved WhatsApp template message."""
        url = f"{self.api_url}/{self.phone_id}/messages"
        headers = {
            "Authorization": f"Bearer {self.access_token}",
            "Content-Type": "application/json"
        }
        payload = {
            "messaging_product": "whatsapp",
            "to": self._format_number(to_number),
            "type": "template",
            "template": {
                "name": template_name,
                "language": {"code": "en"},
                "components": [
                    {
                        "type": "header",
                        "parameters": [{"type": "text", "text": header_vars[0]}]
                    },
                    {
                        "type": "body",
                        "parameters": [
                            {"type": "text", "text": v} for v in body_vars
                        ]
                    }
                ]
            }
        }
        async with httpx.AsyncClient() as client:
            res = await client.post(url, json=payload, headers=headers)
            return res.json()

    def _format_number(self, number: str) -> str:
        """Convert 10-digit Indian number to WhatsApp format."""
        digits = "".join(filter(str.isdigit, number))
        if len(digits) == 10:
```

```
            return "91" + digits
        if digits.startswith("91") and len(digits) == 12:
            return digits
        return digits

    async def send_escalation_alert(
        self, user_name: str, user_phone: str, action_sn: str,
        action_text: str, level: int, hrs_overdue: float
    ) -> dict:
        """Send Level N escalation alert for overdue action."""
        escalation_urgency = {
            1: "⚠️ Level 1",
            2: "🚨 Level 2",
            3: "🔥 Level 3 (Final)"
        }
        header = f"{escalation_urgency.get(level, '⚠️')} Escalation"
        body = [
            f"Hi {user_name},",
            f"Action *{action_sn}* is *{int(hrs_overdue)}h overdue*.",
            f"📋 {action_text[:80]}...",
            f"Please take immediate action or escalate.",
            "— MCS Auto-Alert"
        ]
        return await self.send_template_message(
            user_phone, "escalation_alert_v1", [header], body
        )
```

4.5 Escalation Alert Flow (Updated)



4.6 User Phone Number Management

Add a `phone` field to the Users table:

```
ALTER TABLE users ADD COLUMN phone VARCHAR(15);
-- WhatsApp requires E.164 format: +91XXXXXXXXXX
```

The Master Setup page gets a new "Phone" field for user management.

4.7 WhatsApp Templates (to be pre-approved on Meta Business)

Template Name	Use Case	Variables
`escalation_alert_v1`	Overdue action Level 1	urgency, user_name, action_sn, action_text, hrs_overdue
`escalation_alert_v2`	Overdue action Level 2/3	same + escalation_chain
`action_reminder`	Action due in 24h	action_sn, action_text, due_date
`meeting_reminder`	Meeting starting in 30 min	meeting_title, time, attendees
`action_completed`	Confirmation request sent	action_sn, action_text, requester
`weekly_summary`	Weekly action status digest	open_count, completed_count, overdue_count

4.8 Acceptance Criteria

- [ ] Admin can set WhatsApp phone number per user in Master Setup
- [ ] Escalation Level 1 sends WhatsApp within 5s of trigger (async, non-blocking)
- [ ] If WhatsApp fails, email is sent as fallback (same logic as before)
- [ ] WhatsApp messages are logged in audit trail
- [ ] Template approval status is visible in Admin panel
- [ ] Cost per message tracked and displayed in Dashboard

---

5. Upgrade Module 3 — AI Insight Engine (Gemini + ChatGPT)

5.1 Objective

Replace the current rule-based "insights" system with AI-powered analysis. The meeting transcript should be automatically summarized, action items extracted, risks flagged, and decisions recorded.

5.2 Current State

The current insight system in `MeetingRoom` uses keyword matching:

```
// Current insight logic (simplified)
const detectInsights = (lines) => {
  const insights = [];
  if (lines.some(l => l.toLowerCase().includes("mp below target") || l.toLowerCase().i
    insights.push({ type: "⚠️ Production Gap", text: "Production deviation flagged — c
  if (lines.some(l => l.toLowerCase().includes("safety") || l.toLowerCase().includes("
    insights.push({ type: "🛡️ Safety", text: "Safety concern raised — assign owner" })
  // ... ~10 more hardcoded rules
  return insights;
};
```

Problems:

- Fragile — breaks with synonym variations ("below target" vs "below plan")
- No summarization — user has to read all transcript lines
- No risk scoring — no sense of priority
- No decision recording — decisions mixed with actions

5.3 AI Insight Architecture

Dual-model strategy:

Task	Recommended Model	Reason
Real-time interim insights (during meeting)	**Gemini 1.5 Flash**	Fast, cheap, good at streaming
Post-meeting full summary + action extraction	**GPT-4o-mini**	Best at structured extraction
Domain-specific (ferroalloy/manufacturing)	Fine-tuned model or RAG	Better accuracy on domain terms

5.4 Backend Module: `backend/services/ai_engine.py`

```
from enum import Enum
from typing import Optional
import httpx
import os

class AIModel(str, Enum):
    GEMINI_FLASH = "gemini_flash"
    GPT4O_MINI = "gpt4o_mini"
    CLAUDE_HAIKU = "claude_haiku"

class AIInsightEngine:

    SYSTEM_PROMPT = """You are an expert meeting analyst for a ferroalloy manufacturin
Your role is to analyze meeting transcripts and produce:
1. A 3-bullet executive summary
2. Action items with suggested owners (based on attendee list)
3. Risk flags (any safety, quality, or production concerns)
4. Decisions recorded (explicit choices made during the meeting)
5. A "plant health score" (1-10) based on discussion tone

Be concise, specific, and domain-aware. Use these acronyms:
- MPM = Melting Point Management
- EAF = Electric Arc Furnace
- LRF = Ladle Refining Furnace
- HMS = Heavy Melting Scrap"""

    def __init__(self):
        self.gemini_api_key = os.getenv("GEMINI_API_KEY")
        self.openai_api_key = os.getenv("OPENAI_API_KEY")
        self.anthropic_api_key = os.getenv("ANTHROPIC_API_KEY")

    async def generate_interim_insights(
        self, transcript_lines: list[str], attendee_names: list[str]
    ) -> dict:
        """Real-time insights during meeting — Gemini Flash (fast)."""
        if not transcript_lines:
            return {"summary": [], "flags": [], "score": 5}

        transcript_text = "\n".join(transcript_lines[-20:])  # last 20 lines only
        prompt = f"""Attendees: {', '.join(attendee_names)}

Recent transcript (last 20 lines):
{transcript_text}

Provide a JSON response with:
- "summary": 2 bullet points on what's happening now
- "flags": any risks, gaps, or concerns detected
- "score": plant health score 1-10
- "suggested_action": one concrete next step if anything urgent found"""

        # Use Gemini Flash for speed
```

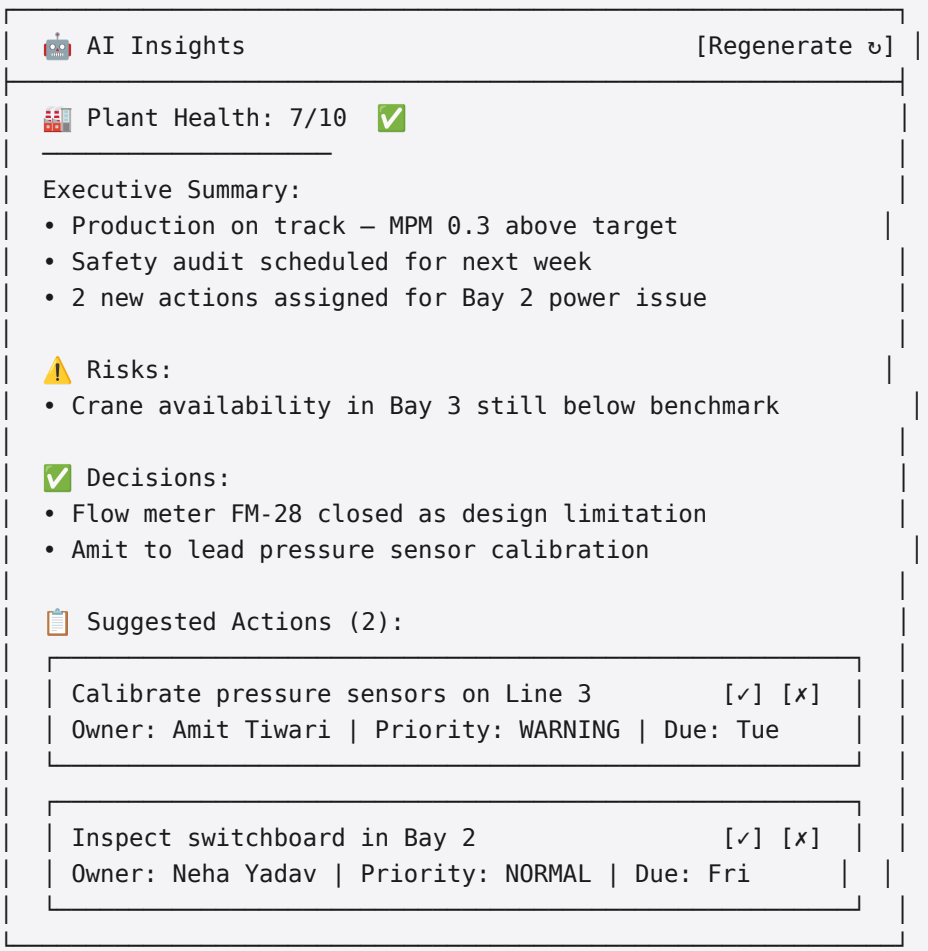
```
        response = await self._call_gemini(prompt)
        return self._parse_json_response(response)

    async def generate_full_meeting_summary(
        self, transcript_lines: list[str], meeting_title: str,
        attendee_names: list[str], meeting_type: str
    ) -> dict:
        """Post-meeting comprehensive analysis — GPT-4o-mini."""
        transcript_text = "\n".join(transcript_lines)
        prompt = f"""MEETING: {meeting_title}
TYPE: {meeting_type}
ATTENDEES: {', '.join(attendee_names)}

TRANSCRIPT:
{transcript_text}

Analyze and return a JSON object with these exact keys:
- "executive_summary": string (3 sentences max)
- "action_items": list of {{"text": str, "owner_suggestion": str, "priority": "CRITICA
- "risks_identified": list of strings
- "decisions_made": list of strings
- "plant_health_score": int 1-10
- "key_takeaways": list of 3 strings
- "escalation_needed": bool
- "escalation_reason": str (if escalation_needed is true)"""

        response = await self._call_openai(prompt)
        return self._parse_json_response(response)

    async def _call_gemini(self, prompt: str) -> str:
        """Call Gemini 1.5 Flash via Google AI Studio API."""
        import google.generativeai as genai
        genai.configure(api_key=self.gemini_api_key)
        model = genai.GenerativeModel("gemini-1.5-flash")
        response = model.generate_content(
            contents=[{
                "role": "user",
                "parts": [{"text": f"{self.SYSTEM_PROMPT}\n\n{prompt}"}]
            }],
            generation_config={"response_mime_type": "text/plain"}
        )
        return response.text

    async def _call_openai(self, prompt: str) -> str:
        """Call GPT-4o-mini via OpenAI API."""
        async with httpx.AsyncClient() as client:
            res = await client.post(
                "https://api.openai.com/v1/chat/completions",
                headers={
                    "Authorization": f"Bearer {self.openai_api_key}",
                    "Content-Type": "application/json"
                },
                json={
                    "model": "gpt-4o-mini",
```

```
                    "messages": [
                        {"role": "system", "content": self.SYSTEM_PROMPT},
                        {"role": "user", "content": prompt}
                    ],
                    "temperature": 0.3,
                    "response_format": {"type": "text"}
                },
                timeout=30.0
            )
            data = res.json()
            return data["choices"][0]["message"]["content"]

    def _parse_json_response(self, raw: str) -> dict:
        """Extract JSON from LLM response (handles markdown code blocks)."""
        import json, re
        # Strip markdown code blocks
        cleaned = re.sub(r"```(?:json)?\s*", "", raw).strip()
        try:
            return json.loads(cleaned)
        except json.JSONDecodeError:
            return {"error": "Parse failed", "raw": cleaned[:500]}
```

5.5 Meeting Summary Storage

```
CREATE TABLE meeting_summaries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    meeting_id UUID REFERENCES meetings(id) ON DELETE CASCADE,
    generated_at TIMESTAMPTZ DEFAULT NOW(),
    model_used TEXT NOT NULL,           -- "gemini_flash" or "gpt4o_mini"
    executive_summary TEXT,
    plant_health_score INT CHECK (plant_health_score BETWEEN 1 AND 10),
    risks_identified JSONB DEFAULT '[]',
    decisions_made JSONB DEFAULT '[]',
    escalation_needed BOOLEAN DEFAULT FALSE,
    escalation_reason TEXT,
    raw_response JSONB,                  -- full raw LLM output for debugging
    cost_usd DECIMAL(8,6) DEFAULT 0     -- API call cost tracking
);

CREATE TABLE extracted_actions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    meeting_id UUID REFERENCES meetings(id) ON DELETE CASCADE,
    raw_text TEXT NOT NULL,
    suggested_owner VARCHAR(100),
    priority VARCHAR(20) CHECK (priority IN ('CRITICAL','WARNING','NORMAL')),
    suggested_due VARCHAR(100),
    confidence_score DECIMAL(3,2),       -- 0.00 to 1.00
    accepted BOOLEAN,                    -- did user accept this suggestion?
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

5.6 User Experience — MeetingRoom Changes



5.7 Acceptance Criteria

- [ ] Real-time interim insights appear during meeting (every ~2 min)
- [ ] Post-meeting summary auto-generates when meeting ends
- [ ] AI-suggested actions can be accepted (→ pre-filled AddActionPanel) or dismissed
- [ ] Plant health score is visible on Dashboard
- [ ] All AI calls are logged with cost tracking
- [ ] If AI service is down, MeetingRoom falls back to rule-based insights (current behavior)
- [ ] Toggle: Admin can disable AI per meeting type in MeetingPresets

---

6. Upgrade Module 4 — Python Backend

6.1 Objective

Replace Google Apps Script + Render.com with a dedicated **FastAPI** backend. This unifies all API logic in one place, enables WebSocket support for real-time features, and gives us full control over authentication, rate limiting, and business logic.

6.2 Why FastAPI

Criteria	FastAPI	Django	Flask
Async support	✅ Native	⚠️ Channels	⚠️ Requires extensions
Auto docs (OpenAPI)	✅ Built-in	✅ Admin panel	❌ Manual
Type safety	✅ Pydantic	✅ ORM	❌ Manual
Deployment	✅ Uvicorn	✅ Gunicorn	✅ Gunicorn
Team familiarity	High (Python)	Medium	High
Real-time (WS)	✅ Native	⚠️ Channels	⚠️ Flask-SocketIO

FastAPI wins: native async, auto-generated OpenAPI docs, Pydantic validation, native WebSocket support, and Python alignment with AI SDKs (Gemini, OpenAI, Anthropic all have Python SDKs).

6.3 Project Structure

```
backend/
├── main.py                      # FastAPI app entry point
├── requirements.txt             # Dependencies
├── config.py                    # Environment variables & config
├── database.py                  # Supabase client + connection pool
│
├── models/                      # Pydantic schemas
│   ├── __init__.py
│   ├── user.py
│   ├── action.py
│   ├── meeting.py
│   ├── escalation.py
│   └── common.py
│
├── routers/                     # API route modules
│   ├── __init__.py
│   ├── auth.py                  # Login, session management
│   ├── actions.py               # CRUD for actions
│   ├── meetings.py              # CRUD + meeting flow
│   ├── users.py                 # User management
│   ├── plants.py                # Plant management
│   ├── dashboard.py             # KPI aggregations
│   ├── escalations.py           # Escalation triggers + alerts
│   ├── master.py                # Admin master data CRUD
│   ├── stt.py                   # WebSocket STT proxy
│   ├── ai.py                    # AI insight endpoints
│   └── whatsapp.py              # WhatsApp test/send endpoint
│
├── services/                    # Business logic
│   ├── __init__.py
│   ├── escalation_engine.py     # Escalation logic (ported from JS)
│   ├── whatsapp_service.py      # WhatsApp Cloud API client
│   ├── ai_engine.py             # AI insight engine (Module 3)
│   ├── email_service.py         # SMTP email (replaces Render.com)
│   ├── stt_router.py            # WebSocket STT routing (Module 1)
│   └── notification_router.py   # Unified notification dispatcher
│
├── middleware/
│   ├── auth.py                  # JWT session validation
│   └── rate_limit.py            # Per-user rate limiting
│
└── scripts/
    ├── seed_data.py             # Initial data seeding
    └── migrate_from_sheets.py   # One-time Google Sheets → Supabase migration
```

6.4 API Design — Key Endpoints

```
# main.py (FastAPI app structure)

from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: initialize Supabase client, load config
    # Shutdown: close DB pool, cleanup WebSocket connections
    yield

app = FastAPI(
    title="MCS API v2.0",
    description="Management Control System Backend",
    version="2.0.0",
    lifespan=lifespan
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # Lock down to known origins in production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ── Auth ──────────────────────────────────────────────────────
@app.post("/api/auth/login")
async def login(req: LoginRequest) -> AuthResponse:
    """Authenticate user against Supabase Auth or Master account."""
    ...

@app.post("/api/auth/logout")
async def logout(user=Depends(get_current_user)):
    ...

# ── Actions ───────────────────────────────────────────────────
@app.get("/api/actions")
async def list_actions(
    plant: str | None = None,
    status: str | None = None,
    priority: str | None = None,
    responsible: str | None = None,
    page: int = 1,
    page_size: int = 50,
    user=Depends(get_current_user)
) -> PaginatedActions:
    ...

@app.post("/api/actions")
```

```
async def create_action(req: CreateActionRequest, user=Depends(get_current_user)) -> A
    ...

@app.patch("/api/actions/{action_id}")
async def update_action(action_id: str, patch: UpdateActionRequest, user=Depends(get_c
    """Update action fields. Triggers revision history if due date changes."""
    ...

@app.post("/api/actions/{action_id}/messages")
async def add_message(action_id: str, req: AddMessageRequest, user=Depends(get_current
    ...

# ── Meetings ──────────────────────────────────────────────────
@app.get("/api/meetings")
async def list_meetings(user=Depends(get_current_user)) -> list[Meeting]:
    ...

@app.post("/api/meetings")
async def create_meeting(req: CreateMeetingRequest, user=Depends(get_current_user)) ->
    ...

@app.patch("/api/meetings/{meeting_id}/commit")
async def commit_meeting(meeting_id: str, actions: list[StagedAction], user=Depends(ge
    """Take staged actions from meeting room and persist as real Actions."""
    ...

# ── Escalations ───────────────────────────────────────────────
@app.post("/api/escalations/trigger")
async def trigger_escalations(user=Depends(get_current_user)):
    """Manually trigger escalation engine. Runs every 15 min via cron."""
    ...

@app.get("/api/escalations/alerts")
async def list_alerts(user=Depends(get_current_user)) -> list[EscalationAlert]:
    ...

# ── Real-time: STT WebSocket ──────────────────────────────────
@app.websocket("/ws/stt/{meeting_id}")
async def stt_websocket(websocket: WebSocket, meeting_id: str, token: str):
    """WebSocket for real-time STT. Streams audio → STT engine → transcript back."""
    await stt_service.handle_connection(websocket, meeting_id, token)

# ── AI Insights ───────────────────────────────────────────────
@app.post("/api/ai/interim-insights")
async def get_interim_insights(req: InterimInsightsRequest, user=Depends(get_current_u
    """Real-time insights during active meeting."""
    ...

@app.post("/api/ai/meeting-summary")
async def get_meeting_summary(req: MeetingSummaryRequest, user=Depends(get_current_use
    """Full AI-powered post-meeting summary."""
    ...

# ── WhatsApp ──────────────────────────────────────────────────
```

```
@app.post("/api/whatsapp/send-test")
async def send_test_message(req: WhatsAppTestRequest, user=Depends(require_admin)):
    """Send a test WhatsApp message to verify setup."""
    ...

# ── Dashboard ──────────────────────────────────────────────────
@app.get("/api/dashboard/kpis")
async def get_kpis(plant: str | None = None, user=Depends(get_current_user)) -> Dashbo
    ...

@app.get("/api/dashboard/trends")
async def get_trends(metric: str, days: int = 30, user=Depends(get_current_user)) -> T
    ...
```

6.5 Authentication

```
# middleware/auth.py
from fastapi import HTTPException, Security
from fastapi.security import HTTPBearer
import jwt, os

security = HTTPBearer()

async def get_current_user(token: str = Security(security)) -> User:
    """Validate JWT token and return user object."""
    try:
        payload = jwt.decode(
            token.credentials,
            os.getenv("JWT_SECRET"),
            algorithms=["HS256"]
        )
        # Fetch full user from Supabase
        user = await supabase.table("users").select("*").eq("id", payload["sub"]).sing
        if not user:
            raise HTTPException(401, "User not found")
        return user
    except jwt.ExpiredSignatureError:
        raise HTTPException(401, "Session expired")
    except jwt.InvalidTokenError:
        raise HTTPException(401, "Invalid token")

def require_admin(user: User = Depends(get_current_user)) -> User:
    if user["role"] != "Admin" and user["id"] != "MASTER":
        raise HTTPException(403, "Admin access required")
    return user
```

6.6 Escalation Engine (Ported from JS)

```
# services/escalation_engine.py

ROLE_HIERARCHY = ["Operator", "Supervisor", "Shift Engineer", "HOD", "Plant Head", "MD

def resolve_escalation_state(action: dict, matrix: list[dict], users: list[dict]) -> d
    """Port of JS resolveEscalationState() to Python."""
    if action["status"] in ("COMPLETED", "DROPPED") or not action.get("due"):
        return None

    now = datetime.now(timezone.utc)
    due = datetime.strptime(action["due"] + "T23:59:59", "%Y-%m-%dT%H:%M:%S").replace(
    hrs_overdue = (now - due).total_seconds() / 3600

    if hrs_overdue < 0:
        return None

    resp_user = next((u for u in users if u["name"] == action["responsible"]), None)
    resp_role = resp_user["role"] if resp_user else action.get("responsible_role", "")

    active_tiers = [
        {**t, "priorities": _norm_p(t["priorities"])}
        for t in matrix if t.get("active")
    ]

    matched = [
        t for t in active_tiers
        if t["from_role"] == resp_role
        and hrs_overdue >= t["overdue_hrs"]
        and action.get("priority", "NORMAL") in t["priorities"]
    ]

    if not matched:
        return None

    matched_tier = sorted(matched, key=lambda t: t["level"])[-1]
    return {
        "tier": matched_tier,
        "hrs_overdue": hrs_overdue,
        "days_overdue": int(hrs_overdue // 24),
        "from_role": resp_role,
    }


async def run_escalation_batch(db, actions: list[dict], users: list[dict]):
    """Port of JS runEscalation() — triggers alerts for all overdue actions."""
    alerts = []

    for action in actions:
        state = resolve_escalation_state(action, ESCALATION_MATRIX, users)
        if not state:
            continue
```

```
        tier = state["tier"]

        # Log to audit
        alert = {
            "id": str(uuid4()),
            "ts": datetime.now(timezone.utc).isoformat(),
            "sn": action["sn"],
            "text": action["text"],
            "level": tier["level"],
            "target_role": tier["target_role"],
            "from_role": state["from_role"],
            "reason": f"{int(state['hrs_overdue'])}h overdue — {tier['label']}",
        }
        alerts.append(alert)

        # Send notifications
        if "WhatsApp" in tier.get("notify_method", ""):
            target_users = [u for u in users if u["role"] == tier["target_role"]]
            for u in target_users:
                if u.get("phone"):
                    await whatsapp_service.send_escalation_alert(
                        user_name=u["name"],
                        user_phone=u["phone"],
                        action_sn=action["sn"],
                        action_text=action["text"],
                        level=tier["level"],
                        hrs_overdue=state["hrs_overdue"],
                    )

        if "Email" in tier.get("notify_method", ""):
            await email_service.send_escalation_email(
                to_user=target_users[0] if target_users else None,
                action=action,
                tier=tier,
                hrs_overdue=state["hrs_overdue"],
            )

    # Persist alerts to audit table
    if alerts:
        await db.table("audit_log").insert(alerts).execute()

    return alerts
```

6.7 Cron Jobs

```
# main.py — background tasks
from fastapi_utils.tasks import repeat_every

@app.on_event("startup")
@repeat_every(seconds=900)  # Every 15 minutes
async def escalation_cron():
    """Run escalation engine every 15 minutes."""
    async with get_db() as db:
        actions = await db.table("actions").select("*").execute()
        users = await db.table("users").select("*").execute()
        await run_escalation_batch(db, actions.data, users.data)
```

6.8 Acceptance Criteria

- [ ] All current Google Apps Script endpoints are replaced with FastAPI equivalents
- [ ] API has OpenAPI documentation at `/docs` (auto-generated)
- [ ] JWT-based auth replaces localStorage session (more secure)
- [ ] Escalation engine runs on a cron schedule, not just on page load
- [ ] WebSocket endpoint for STT streaming is functional
- [ ] All endpoints return proper HTTP status codes + error messages
- [ ] Rate limiting: 100 req/min per user, 1000 req/min per admin
- [ ] Health check endpoint: `GET /health`

---

7. Upgrade Module 5 — Supabase Database Migration

7.1 Objective

Migrate all data from Google Sheets to **Supabase** (PostgreSQL). Supabase gives us a real relational DB with Row Level Security (RLS), real-time subscriptions, and a built-in Auth system — all on a generous free tier.

7.2 Why Supabase

Criteria	Google Sheets	Supabase
Data integrity	❌ No constraints	✅ ACID transactions, FK, checks
Concurrent writes	❌ Throttles at ~60 writes/min	✅ 1000s concurrent
Row-level security	❌ No	✅ RLS per user/role
Real-time subscriptions	❌ No	✅ Built-in WebSocket
Auth	❌ Manual username/password in sheet	✅ Supabase Auth (email, magic link, OAuth)
Free tier	✅ Free	✅ 500MB DB, 1GB transfer/mo
Storage	❌ None	✅ 1GB file storage
Edge functions	❌ No	✅ Deno/TypeScript edge functions
Backups	❌ Manual	✅ Automated daily

Not switching to: Firebase (vendor lock-in), PlanetScale (no foreign keys), MongoDB (no schema = harder to maintain).

7.3 Database Schema

```
-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- ── Enums ──────────────────────────────────────────────────────
CREATE TYPE action_status AS ENUM ('NOT STARTED', 'IN PROCESS', 'COMPLETED', 'DROPPED'
CREATE TYPE priority AS ENUM ('CRITICAL', 'WARNING', 'NORMAL');

-- ── Users ──────────────────────────────────────────────────────
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255),              -- NULL for SSO users
    name VARCHAR(100) NOT NULL,
    role VARCHAR(30) NOT NULL DEFAULT 'Guest User',
    plant VARCHAR(100),
    dept VARCHAR(100),
    superior VARCHAR(100),
    email VARCHAR(150),
    phone VARCHAR(15),                       -- WhatsApp number
    initials VARCHAR(3),
    color VARCHAR(10),
    is_master BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Plants ─────────────────────────────────────────────────────
CREATE TABLE plants (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(100) UNIQUE NOT NULL,
    code VARCHAR(20),
    location VARCHAR(200),
    manager VARCHAR(100),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Departments ────────────────────────────────────────────────
CREATE TABLE departments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(100) NOT NULL,
    plant_id UUID REFERENCES plants(id) ON DELETE SET NULL,
    hod VARCHAR(100),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Roles ──────────────────────────────────────────────────────
CREATE TABLE roles (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(50) UNIQUE NOT NULL,
    level INT NOT NULL CHECK (level BETWEEN 1 AND 10),
    description TEXT,
```

```
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Actions ──────────────────────────────────────────────────
CREATE TABLE actions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    sn VARCHAR(20) UNIQUE NOT NULL,         -- "ACT-001"
    text TEXT NOT NULL,
    responsible VARCHAR(100) NOT NULL,
    responsible_role VARCHAR(50),
    status action_status DEFAULT 'NOT STARTED',
    priority priority DEFAULT 'NORMAL',
    due DATE,
    plant VARCHAR(100),
    dept VARCHAR(100),
    section VARCHAR(50),
    machine VARCHAR(100),
    source_meeting VARCHAR(20),             -- meeting SN reference
    source VARCHAR(50) DEFAULT 'Manual',
    assigned_by VARCHAR(100),
    allocated_by VARCHAR(100),
    date_of_action DATE DEFAULT CURRENT_DATE,
    created DATE DEFAULT CURRENT_date,
    closed_on DATE,
    revisions INT DEFAULT 0,
    revision_history JSONB DEFAULT '[]',   -- [{date, from, to, by}]
    pending_confirmation BOOLEAN DEFAULT FALSE,
    messages JSONB DEFAULT '[]',            -- [{id, by, text, ts}]
    revision_locked BOOLEAN DEFAULT FALSE,
    reason VARCHAR(200),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Index for common queries
CREATE INDEX idx_actions_status ON actions(status);
CREATE INDEX idx_actions_responsible ON actions(responsible);
CREATE INDEX idx_actions_due ON actions(due);
CREATE INDEX idx_actions_plant ON actions(plant);
CREATE INDEX idx_actions_priority ON actions(priority);

-- ── Meetings ─────────────────────────────────────────────────
CREATE TABLE meetings (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    sn VARCHAR(20) UNIQUE NOT NULL,
    title VARCHAR(300) NOT NULL,
    meeting_type VARCHAR(100),
    date DATE NOT NULL,
    time TIME,
    duration_minutes INT,
    plant VARCHAR(100),
    attendees JSONB DEFAULT '[]',           -- ["Anil Kumar", "Meena Joshi"]
    attendees_finalized BOOLEAN DEFAULT FALSE,
    project_id UUID REFERENCES projects(id) ON DELETE SET NULL,
```

```
    instructions JSONB DEFAULT '[]',
    status VARCHAR(30) DEFAULT 'PLANNED',    -- PLANNED | IN_PROGRESS | COMPLETED
    transcript JSONB DEFAULT '[]',          -- [{line_no, text, ts}]
    started_at TIMESTAMPTZ,
    ended_at TIMESTAMPTZ,
    created_by VARCHAR(100),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Projects ──────────────────────────────────────────────────
CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    sn VARCHAR(20) UNIQUE NOT NULL,
    name VARCHAR(300) NOT NULL,
    plant VARCHAR(100),
    dept VARCHAR(100),
    status VARCHAR(30) DEFAULT 'Active',
    priority priority DEFAULT 'NORMAL',
    start_date DATE,
    target_date DATE,
    description TEXT,
    created_by VARCHAR(100),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Escalation Matrix ─────────────────────────────────────────
CREATE TABLE escalation_matrix (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    sn VARCHAR(20) UNIQUE NOT NULL,
    level INT NOT NULL,
    from_role VARCHAR(50) NOT NULL,
    target_role VARCHAR(50) NOT NULL,
    label VARCHAR(200),
    overdue_days INT,
    overdue_hrs INT NOT NULL,
    notify_method VARCHAR(100),
    priorities JSONB DEFAULT '["CRITICAL","WARNING","NORMAL"]',
    applicable_to VARCHAR(50) DEFAULT 'All',
    color VARCHAR(10),
    active BOOLEAN DEFAULT TRUE,
    description TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Machines ──────────────────────────────────────────────────
CREATE TABLE machines (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(100) NOT NULL,
    plant VARCHAR(100),
    section VARCHAR(100),
    machine_type VARCHAR(100),
    status VARCHAR(30) DEFAULT 'Operational',
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- ── Meeting Presets ──────────────────────────────────────────
CREATE TABLE meeting_presets (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    meeting_type VARCHAR(100) UNIQUE NOT NULL,
    attendees JSONB DEFAULT '[]',
    instructions JSONB DEFAULT '[]',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Reasons ──────────────────────────────────────────────────
CREATE TABLE reasons (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    label VARCHAR(200) NOT NULL,
    category VARCHAR(50),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Audit Log ────────────────────────────────────────────────
CREATE TABLE audit_log (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    sn VARCHAR(20) NOT NULL,
    action_text TEXT,
    level INT,
    target_role VARCHAR(50),
    from_role VARCHAR(50),
    reason TEXT,
    ts TIMESTAMPTZ DEFAULT NOW(),
    notification_sent VARCHAR(20) DEFAULT 'pending',
    notification_channel VARCHAR(50)
);

CREATE INDEX idx_audit_sn_level ON audit_log(sn, level);

-- ── Meeting Summaries (AI) ──────────────────────────────────
CREATE TABLE meeting_summaries (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    meeting_id UUID REFERENCES meetings(id) ON DELETE CASCADE,
    model_used VARCHAR(50) NOT NULL,
    executive_summary TEXT,
    plant_health_score INT CHECK (plant_health_score BETWEEN 1 AND 10),
    key_takeaways JSONB DEFAULT '[]',
    risks_identified JSONB DEFAULT '[]',
    decisions_made JSONB DEFAULT '[]',
    escalation_needed BOOLEAN DEFAULT FALSE,
    escalation_reason TEXT,
    raw_response JSONB,
    cost_usd DECIMAL(8,6) DEFAULT 0,
    generated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Extracted Actions (AI) ──────────────────────────────────
CREATE TABLE extracted_actions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    meeting_id UUID REFERENCES meetings(id) ON DELETE CASCADE,
    raw_text TEXT NOT NULL,
    suggested_owner VARCHAR(100),
    priority VARCHAR(20),
    suggested_due VARCHAR(100),
    confidence_score DECIMAL(3,2),
    accepted BOOLEAN,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Notification Log ──────────────────────────────────────
CREATE TABLE notification_log (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    channel VARCHAR(20) NOT NULL,          -- 'whatsapp', 'email', 'in_app'
    template VARCHAR(50),
    payload JSONB,
    status VARCHAR(20) DEFAULT 'sent',     -- 'sent', 'failed', 'delivered', 'read'
    external_id VARCHAR(200),              -- WhatsApp message ID
    cost_usd DECIMAL(8,6) DEFAULT 0,
    sent_at TIMESTAMPTZ DEFAULT NOW()
);

-- ── Row Level Security ────────────────────────────────────
ALTER TABLE actions ENABLE ROW LEVEL SECURITY;
ALTER TABLE meetings ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

-- Actions: users see all if Admin/MD/PlantHead, else only their plant's actions
CREATE POLICY "actions_read_policy" ON actions FOR SELECT
    USING (
        (SELECT role FROM users WHERE users.id = auth.uid()) IN ('Admin', 'MD', 'Plant
        OR plant = (SELECT plant FROM users WHERE users.id = auth.uid())
        OR responsible = (SELECT name FROM users WHERE users.id = auth.uid())
    );

CREATE POLICY "actions_write_policy" ON actions FOR ALL
    USING (
        (SELECT role FROM users WHERE users.id = auth.uid()) IN ('Admin', 'MD', 'Plant
    );

-- Meetings: read all if admin/plant head, else own meetings or plant meetings
CREATE POLICY "meetings_read_policy" ON meetings FOR SELECT
    USING (
        created_by = (SELECT name FROM users WHERE users.id = auth.uid())
        OR plant = (SELECT plant FROM users WHERE users.id = auth.uid())
        OR (SELECT role FROM users WHERE users.id = auth.uid()) = 'Admin'
    );

ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;
CREATE POLICY "audit_read_policy" ON audit_log FOR SELECT
    USING (
```

```
          (SELECT role FROM users WHERE users.id = auth.uid()) IN ('Admin', 'MD', 'Plant
    );
```

7.4 Supabase Realtime (取代 Sheet polling)

Replace 60-second Sheet polling with Supabase Realtime subscriptions:

```
# frontend/hooks/useRealtimeData.js
import { supabase } from '@/lib/supabase'

export function useRealtimeActions(user, setActions) {
  useEffect(() => {
    const channel = supabase
      .channel('actions_changes')
      .on(
        'postgres_changes',
        {
          event: '*',
          schema: 'public',
          table: 'actions',
          filter: user?.plant ? `plant=eq.${user.plant}` : undefined
        },
        (payload) => {
          if (payload.eventType === 'INSERT') {
            setActions(prev => [...prev, payload.new])
          } else if (payload.eventType === 'UPDATE') {
            setActions(prev => prev.map(a => a.id === payload.new.id ? payload.new : a
          } else if (payload.eventType === 'DELETE') {
            setActions(prev => prev.filter(a => a.id !== payload.old.id))
          }
        }
      )
      .subscribe()

    return () => { supabase.removeChannel(channel) }
  }, [user?.plant])
}
```

7.5 One-Time Migration: Google Sheets → Supabase

```
# backend/scripts/migrate_from_sheets.py

"""
One-time script to migrate all data from Google Sheets to Supabase.
Run once during the upgrade window, then never again.

Steps:
1. Read all tabs from the source Google Sheet
2. Transform rows to match Supabase schema
3. Insert into Supabase (upsert by ID/SN to be idempotent)
4. Verify row counts match
5. Archive the old Google Sheet (read-only)
"""

import asyncio
import gspread
from supabase import create_client, Client
from dotenv import load_dotenv
import os

load_dotenv()

SHEET_ID = os.getenv("GOOGLE_SHEET_ID")
SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_KEY = os.getenv("SUPABASE_SERVICE_ROLE_KEY")

async def migrate_tab(tab_name: str, supabase: Client, gc):
    """Migrate one sheet tab to Supabase."""
    ws = gc.open_by_key(SHEET_ID).worksheet(tab_name)
    rows = ws.get_all_records()

    if not rows:
        print(f"  ✓ {tab_name}: empty, skipping")
        return

    # Transform + clean
    cleaned = [clean_row(r, tab_name) for r in rows]
    cleaned = [r for r in cleaned if r]  # drop empty

    # Upsert (replace existing by ID or SN)
    table_map = {
        "Users": "users",
        "Actions": "actions",
        "Meetings": "meetings",
        "Plants": "plants",
        "Departments": "departments",
        "Projects": "projects",
        "EscalationMatrix": "escalation_matrix",
        "Machines": "machines",
        "MeetingPresets": "meeting_presets",
        "Reasons": "reasons",
```

```
        "Roles": "roles",
    }

    table_name = table_map.get(tab_name)
    if not table_name:
        print(f"  ? {tab_name}: no table mapping, skipping")
        return

    result = supabase.table(table_name).upsert(cleaned).execute()
    print(f"  ✓ {tab_name}: {len(cleaned)} rows → Supabase '{table_name}'")

async def main():
    gc = gspread.service_account()  # needs service account JSON
    sb = create_client(SUPABASE_URL, SUPABASE_KEY)

    print("Starting Google Sheets → Supabase migration\n")

    tabs = ["Users", "Plants", "Departments", "Actions", "Meetings",
            "Projects", "EscalationMatrix", "Machines", "MeetingPresets",
            "Reasons", "Roles", "Audit"]

    for tab in tabs:
        await migrate_tab(tab, sb, gc)

    # Verify counts
    for table in ["users", "plants", "actions", "meetings"]:
        count = sb.table(table).select("id", count="exact").execute()
        print(f"  → {table}: {count.count} rows")

    print("\n✅ Migration complete. Old Google Sheet should now be archived to read-on

if __name__ == "__main__":
    asyncio.run(main())
```

7.6 Acceptance Criteria

- [ ] All Google Sheet tabs migrated to Supabase with correct schema
- [ ] Row counts match between old sheet and new DB (verified in migration script)
- [ ] Foreign key relationships enforced (no orphan rows)
- [ ] RLS policies prevent users from seeing other plants' data
- [ ] Real-time updates work across multiple browser tabs
- [ ] Google Sheet is archived (read-only) — never used as source again
- [ ] Supabase backup verified (can restore to point-in-time)

---

8. Migration Strategy

8.1 Parallel Operation (Recommended for Safety)

8.2 Data Integrity Checks

```
-- Run after migration to verify integrity
SELECT
  (SELECT COUNT(*) FROM actions) as action_count,
  (SELECT COUNT(*) FROM actions WHERE status = 'COMPLETED') as completed,
  (SELECT COUNT(*) FROM actions WHERE due < CURRENT_DATE AND status NOT IN ('COMPLETED

SELECT
  (SELECT COUNT(DISTINCT responsible) FROM actions) as unique_responsibles,
  (SELECT COUNT(DISTINCT plant) FROM actions) as unique_plants;

-- Check for duplicate SNs (should be 0)
SELECT sn, COUNT(*) FROM actions GROUP BY sn HAVING COUNT(*) > 1;
```

---

9. Testing Plan

9.1 Unit Tests (Python Backend)

```
# backend/tests/test_escalation_engine.py
import pytest
from services.escalation_engine import resolve_escalation_state, DEFAULT_ESC_MATRIX

class TestEscalationEngine:
    def test_overdue_action_triggers_level_1(self):
        action = {
            "sn": "ACT-001",
            "text": "Calibrate sensors",
            "status": "IN PROCESS",
            "due": "2025-07-01",  # 6 days overdue from July 7
            "responsible": "Amit Tiwari",
            "priority": "CRITICAL",
        }
        users = [{"name": "Amit Tiwari", "role": "Operator"}]

        state = resolve_escalation_state(action, DEFAULT_ESC_MATRIX, users)
        assert state is not None
        assert state["tier"]["level"] == 1

    def test_completed_action_never_escalates(self):
        action = {
            "sn": "ACT-001",
            "status": "COMPLETED",
            "due": "2020-01-01",
            "priority": "CRITICAL",
        }
        state = resolve_escalation_state(action, DEFAULT_ESC_MATRIX, [])
        assert state is None

    def test_future_due_action_never_escalates(self):
        action = {
            "sn": "ACT-001",
            "status": "IN PROCESS",
            "due": "2030-01-01",
            "priority": "CRITICAL",
        }
        state = resolve_escalation_state(action, DEFAULT_ESC_MATRIX, [])
        assert state is None
```

9.2 Integration Tests

Test	Method	Success Criteria
Login flow	POST `/api/auth/login` with correct creds	Returns JWT, user object
Create action	POST `/api/actions` + GET `/api/actions/{id}`	Round-trip matches
Meeting commit	POST `/api/meetings/{id}/commit`	All staged actions appear in actions table
Escalation trigger	Cron fires `run_escalation_batch`	Audit rows inserted, WhatsApp API called
AI summary	POST `/api/ai/meeting-summary`	Returns structured JSON with all required keys
Real-time sync	Two tabs open, update action in Tab 1	Tab 2 reflects change within 2s

9.3 Load Testing

```
# backend/tests/load_test.py
import httpx
import asyncio
from datetime import datetime

async def test_concurrent_users():
    """Simulate 100 users hitting /api/actions simultaneously."""
    async with httpx.AsyncClient(base_url="http://localhost:8000") as client:
        tasks = []
        for i in range(100):
            token = await get_test_token(client)  # login
            tasks.append(client.get(
                "/api/actions?page=1&page_size=50",
                headers={"Authorization": f"Bearer {token}"}
            ))
        responses = await asyncio.gather(*tasks, return_exceptions=True)
        successes = [r for r in responses if not isinstance(r, Exception) and r.status
        print(f"✅ {len(successes)}/100 requests succeeded in < 500ms")
```

Target: **100 concurrent users, < 200ms p95 response time on list endpoints.**

---

10. Rollback Plan

10.1 Rollback Trigger Conditions

Initiate rollback if any of these occur within 72 hours of go-live:

Condition	Threshold
API error rate	> 5% of requests returning 5xx
Database row count mismatch	> 1% difference from pre-migration baseline
Supabase outage	Any unplanned downtime > 5 minutes
WhatsApp delivery rate	< 80% of messages delivered within 5 min
AI service uptime	< 95% (allow fallback to rule-based)

10.2 Rollback Procedure

```
1. Revert DNS/env vars to point frontend at old Google Apps Script URLs
2. Keep Supabase running in read-only mode (for audit purposes)
3. Old Google Sheet: remove read-only lock, restore write access
4. Notify users: "System restored to previous version"
5. Post-mortem: document root cause, fix, re-schedule go-live
```

10.3 Rollback Time Estimate

- DNS/env change: 5 minutes
- Old sheet write access restoration: 2 minutes
- **Total RTO (Recovery Time Objective): ~10 minutes**

---

11. Timeline & Phasing

Recommended 14-Week Plan

Week	Phase	Deliverables
1-2	**Foundation**	Supabase project setup, schema creation, RLS policies, migration script
3-4	**Backend Core**	FastAPI skeleton, all CRUD endpoints, JWT auth, Supabase client
5-6	**Escalation Engine**	Port JS escalation logic, WhatsApp service, email service, cron jobs
7-8	**AI Integration**	AI Insight Engine, meeting summary endpoint, frontend AI panel
9	**STT Module**	WebSocket STT proxy, Deepgram integration, frontend streaming
10	**Migration**	Run migration script, validate data, shadow mode testing
11	**UAT**	User acceptance testing with power users, bug fixes
12	**Go Live**	Cutover, 24h monitoring, rollback on standby
13-14	**Stabilization**	Monitor KPIs, address edge cases, full documentation

Dependencies

```
Supabase setup (Wk 1)
    └─► Backend Core (Wk 3) — depends on schema
            └─► Escalation Engine (Wk 5) — uses backend auth + DB
            └─► AI Integration (Wk 7) — uses backend endpoints
            └─► STT Module (Wk 9) — uses backend WebSocket
    └─► Migration (Wk 10) — depends on schema + backend
    └─► UAT (Wk 11) — depends on all above
```

---

12. Budget Estimates

Monthly Cost (v2.0 — Production)

Service	Plan	Monthly Cost	Notes
Supabase	Pro Plan	$25/mo	8GB RAM, 100GB storage, real-time
FastAPI Backend	Render.com (2 instances)	$20/mo	Or Railway $15/mo, or Fly.io free tier
Deepgram STT	Pay-as-you-go	~$10-50/mo	~$0.004/min, ~2500 meeting-minutes
WhatsApp Cloud API	Pay-as-you-go	~$5-30/mo	~$0.30/msg, 100-500 alerts/mo
OpenAI GPT-4o-mini	Pay-as-you-go	~$5-20/mo	~$0.15/1M tokens, ~50 summaries/mo
Google Gemini	Free tier	$0	15 req/min, sufficient for interim insights
Domain + SSL	~	$5/mo	
Monitoring (Sentry)	$0	Free tier	Error tracking
CI/CD (GitHub Actions)	$0	Free tier	
Total		**~$70-150/mo**	

vs. Current: ~$0-5/mo (Google Sheet is free, Render.com free tier).

One-Time Costs

Item	Cost
WhatsApp Business verification	~₹3,000-15,000 (varies by country)
WhatsApp template approval (6 templates)	~₹5,000 legal/compliance setup
STT vendor accounts setup	~$0 (Deepgram has free trial)
Migration engineering	~40-80 dev hours
Total one-time	**~$10,000-25,000**

13. Open Risks

13.1 Technical Risks

Risk	Likelihood	Impact	Mitigation
Google Sheets data doesn't map cleanly to relational schema	Medium	High	Spend Week 1 on schema design + test migration with real data; keep type-flexible columns
WhatsApp Cloud API templates rejected by Meta	Medium	Medium	Submit templates early (Week 2); use 3 fallback email templates until approved
AI hallucinating action items	Low	Medium	Low temperature (0.3), validate with confidence score; user must accept/reject each suggestion
Supabase Realtime WebSocket disconnects on poor networks	Medium	Low	Auto-reconnect with exponential backoff; show "reconnecting" indicator
STT quality worse than Chrome native in Hindi-English code-mixing	Medium	Medium	Allow per-meeting STT engine selection; default to Deepgram (good code-mixing support)

13.2 Business Risks

Risk	Mitigation
Users resistant to change (Sheet was simple)	Gradual rollout; keep core UX identical; UAT with real users before go-live
WhatsApp number mismatch (users enter wrong number)	Validate phone number format on save; send verification OTP before first alert
Google Sheet data loss during migration	Full backup before migration; keep sheet read-only for 30 days post-migration
STT costs spiral if meetings run long	Per-meeting cost cap; alert admin if >₹500/mo; fallback to rule-based insights

13.3 Security Considerations

- **JWT secret**: Store in environment variable, rotate every 90 days
- **Supabase service role key**: Never exposed to frontend — server-side only

- **WhatsApp access token**: Refresh monthly; use webhook verification to detect leaks
- **AI API keys**: Never log raw transcripts; redact before sending to AI
- **RLS policies**: Test with non-admin user before go-live — ensure cross-plant isolation
- **HTTPS everywhere**: Backend on HTTPS, Supabase enforces HTTPS, STT WebSocket over WSS

Appendix A — Environment Variables

```
# backend/.env

# Supabase
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...   # server-side only, never frontend

# Auth
JWT_SECRET=your-256-bit-secret-here
JWT_ALGORITHM=HS256
JWT_EXPIRY_HOURS=720   # 30 days

# AI Services
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GEMINI_API_KEY=AIza...

# STT
DEEPGRAM_API_KEY=...
# Or for self-hosted Whisper:
WHISPER_API_URL=http://localhost:8001/transcribe

# WhatsApp (Meta Cloud API)
WHATSAPP_PHONE_ID=123456789
WHATSAPP_ACCESS_TOKEN=EAA...
WHATSAPP_WEBHOOK_VERIFY_TOKEN=your-verify-token
WHATSAPP_API_VERSION=v18.0

# Email (SMTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=alerts@mcs-platform.com
SMTP_PASSWORD=app-specific-password
EMAIL_FROM="MCS Alerts <alerts@mcs-platform.com>"

# External API (for future integrations)
EXTERNAL_API_KEY=...

# App config
APP_ENV=production
LOG_LEVEL=INFO
CORS_ORIGINS=https://mcs.yourcompany.com
```

Appendix B — Frontend Changes Overview

The React frontend needs these key changes to connect to the new backend:

```
// New: src/lib/api.js
import { createClient } from '@supabase/supabase-js'

export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
)

export const API_BASE = import.meta.env.VITE_API_BASE_URL  // FastAPI server

export async function api(endpoint, options = {}) {
  const token = localStorage.getItem('mcs_jwt')
  const res = await fetch(`${API_BASE}${endpoint}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { 'Authorization': `Bearer ${token}` } : {}),
      ...options.headers,
    },
  })
  if (!res.ok) throw new Error(await res.text())
  return res.json()
}

// Replace all sheetGet() calls with:
export const getActions = (params) => api('/api/actions', { params })
export const createAction = (data) => api('/api/actions', { method: 'POST', body: JSON
// etc.
```

```
// MeetingRoom — WebSocket for STT
// Replace SpeechRecognition with WebSocket streaming

const [ws, setWs] = useState(null)

const startStreaming = (meetingId) => {
  const token = localStorage.getItem('mcs_jwt')
  const socket = new WebSocket(
    `${WSS_BASE}/ws/stt/${meetingId}?token=${token}`
  )

  socket.onmessage = (e) => {
    const data = JSON.parse(e.data)
    if (data.type === 'transcript') {
      addTranscriptLine(data.text, data.is_final)
    }
  }

  // Stream microphone to WebSocket
  const mediaRecorder = new MediaRecorder(stream, { mimeType: 'audio/webm' })
  mediaRecorder.ondataavailable = (e) => {
    if (e.data.size > 0) socket.send(e.data)
  }
  mediaRecorder.start(500)  // chunk every 500ms

  setWs(socket)
  return () => {
    mediaRecorder.stop()
    socket.close()
  }
}
```

End of Upgrade Specification — MCS v2.0 Maintainer: Development Team Version: 2.0-DRAFT

— end of document —